Exercise - Power analysis

This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex10
sh > cd ex10
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex10/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex10
```

Student Task 0

Additionally, we need to initialize the croc directory with the cockpit tool to set up all technology related files:

```
sh> cd croc
sh> icdesign ihp13 -update all -nogui
```

Learning Objectives

- Understanding power dissipation in CMOS circuits.
- How to get node activity figures with adequate accuracy?
- How to estimate the circuit power from the node activity figures?
- How to locate IR drop in power and ground networks?
- How can power and ground distribution networks be improved to mitigate IR drop?

Exercises

Theoretical Background on CMOS Circuit Power

Modern CMOS circuits consume power through multiple mechanisms, but the largest contributor in typical digital logic is dynamic power—the energy required to switch logic gates from one state to another. While there are also important sources of static power (mainly due to leakage currents) and short-circuit (or crossover) power, those effects are usually smaller in well-designed CMOS processes, especially under nominal voltages and conservative fabrication technologies. In this exercise, we primarily focus on dynamic power and how to estimate it from switching activities and circuit capacitances.

Sources of Power Dissipation in CMOS

In static CMOS circuits, four main phenomena cause energy dissipation:

- *Charging and Discharging of Node Capacitances* (Dynamic)
- *Crossover (Short-Circuit) Currents* (Dynamic)
- *Resistive Loads* (Static; usually negligible in pure CMOS designs)
- *Leakage Currents* (Static; can become significant in advanced low-voltage processes)

We will not be concerned with static power in this exercise as we limit ourselves to pure CMOS circuits with no resistive loads and because leakage is almost negligible due to the conservative fabrication process being studied. For the needs of EDA tools the dynamic dissipation can be attributed to library cells as follows.

- **Internal Power** (P_{int}): The energy dissipated in the internal nodes of a standard cell due to driving internal transistors and internal node parasitics. This also covers short-circuit currents internal to each gate.
- **Switching Power** (P_{ext}): The power associated with charging and discharging the external load, namely the parasitic interconnect capacitances and the input gate capacitances of any cells connected to the output net.

Hence, the total dynamic power for a single cell can be approximated as:

$$P_{\text{dyn}} = P_{\text{int}} + P_{\text{ext}} \, (1)$$

$$P_{\text{ext}} = f_{\text{cp}} \frac{\alpha_{\text{net}}}{2} C_{\text{ext}} V_{\text{dd}}^2 \, (2)$$

where α_{net} denotes the switching activity of the cell's output node, C_{ext} the load capacitance attached, and V_{dd}^2 the supply voltage. f_{cp} stands for the computation rate, i.e., the inverse of the computation period. P_{int} gets calculated in a similar way, yet coming up with accurate activity and capacitance figures requires detailed information about the inner circuitry and layout of each cell.

When neglecting static power, the overall chip power is obtained by summing the dynamic contributions of all cells and nets:

$$P_{\text{tot}} = P_{\text{stat}} + P_{\text{dyn}} \approx P_{\text{dyn}} = P_{\text{int}} + P_{\text{ext}} \, (3)$$

P_{int} gets calculated in a similar way, yet coming up with accurate activity and capacitance figures requires detailed information about the inner circuitry and layout of each cell.

A power estimator is essentially a piece of software that sums up the various contributions over an entire circuit. Provided the same clock and voltage are used everywhere, this amounts to:

$$P_{\text{ckt}} = \sum_{m=1}^{M} P_{\text{int},m} + \sum_{n=1}^{N} P_{\text{ext},n} \approx f_{\text{cp}} V_{\text{dd}}^2 \left(\sum_{m=1}^{M} \frac{\alpha_m}{2} C_{\text{int},m} + \sum_{n=1}^{N} \frac{\alpha_n}{2} C_{\text{ext},n} \right) \quad (4)$$

Index $m = 1, \ldots, M$ refers to the cells instantiated in the circuit, and $n = 1, \ldots, N$ to the nets of interconnect running in between. For each cell, an internal activity figure α_m is estimated from the node activities at the input(s). Note that $C_{\text{int},m}$ is not meant to correspond to any capacitance physically present in the circuit. Rather, it is just a numerical parameter adjusted for each cell during library characterization to model its internal dissipation.

$Equation(4)$ tells us a few important things about power dissipation and power estimation:

- Accurate Node Activity: Obtaining reliable per-net switching activity (α) is crucial. Realistic simulations—particularly post-layout netlist simulations with extracted parasitics—help capture actual switching behavior.
- Layout Parasitics: Proper extraction of interconnect capacitances is needed to track the real capacitive load on each net.
- Voltage and Frequency Scaling: Dynamic power depends quadratically on V_{dd} and linearly on f_{cp}. Reducing supply voltage or clock frequency significantly decreases dynamic power but can impact performance.
- Clock Gating and Architectural Optimizations: Since dynamic power is consumed primarily when signals switch, gating unused logic (especially clock signals) or disabling large functional blocks when inactive can lead to major power savings.

Manual Activity and Power Calculations for Warm up

Circuit Overview

Suppose we have a simple arithmetic circuit that operates on two 4-bit unsigned inputs *a_i* and *b_i*. A single-bit control *add_i* switches between addition and multiplication, and a multiplexer selects the output of either the adder or the multiplier into an 8-bit output register, clocked by *clk_i*. The circuit is shown in Figure 1, following the systemverilog logic:

```
module example_alu (
  input  logic       clk_i,
  input  logic       add_i,
  input  logic [3:0] a_i,
  input  logic [3:0] b_i,
  output logic [7:0] output_o
);

  logic [7:0] add_res, mul_res;

  assign add_res = a_i + b_i;
  assign mul_res = a_i * b_i;

  always_ff @(posedge clk_i) begin
    if (add_i)
      output_o <= add_res;
    else
      output_o <= mul_res;
  end
endmodule
```

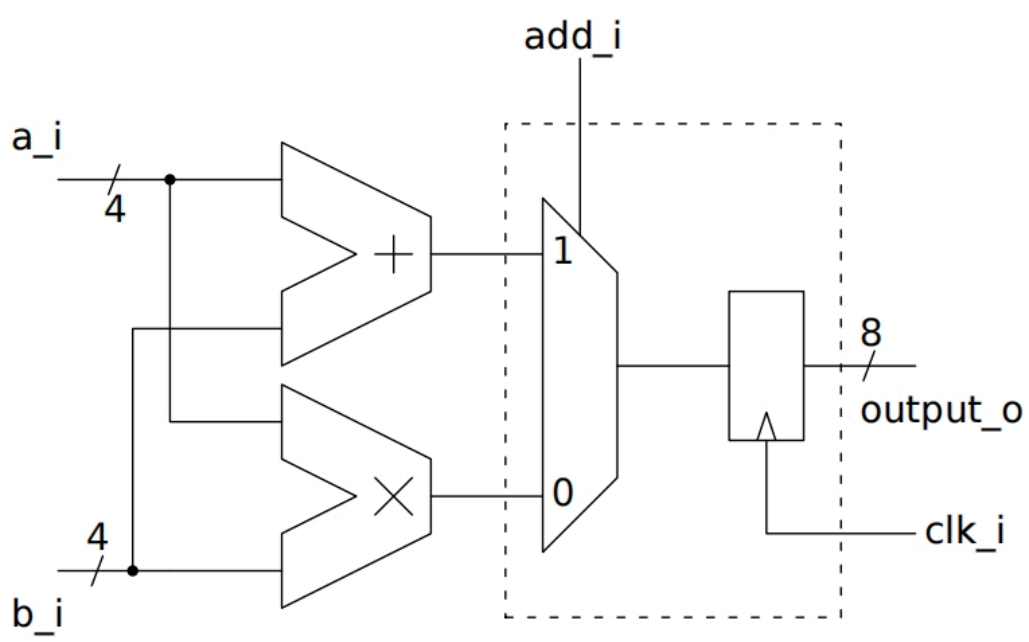

Figure 1: A small arithmetic unit used for hand calculations

Waveforms

An example of signal transitions is shown below. We assume the frequency of *clk_i* is $f_{\text{clk}} = 200\ \text{MHz}$ and the input waveforms are represented in Figure 2. They are periodic and the two input values (*a_i* and *b_i*) have always been chosen to be the same. Moreover, assume that no glitches occur and that the supply voltage $V_{\text{dd}} = 1.2\ \text{V}$.

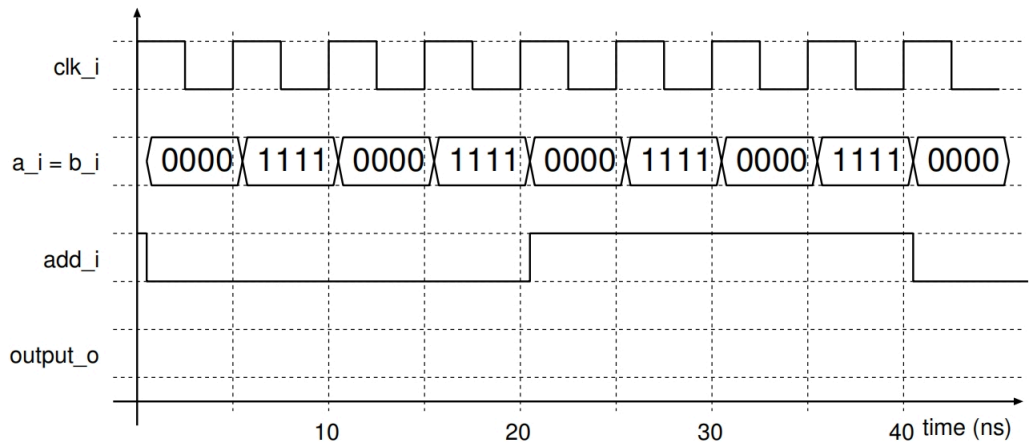

Figure 2: Input and output waveforms

Net	Capacitive load C_{ext} [fF]	Node activity α [1]	Switching power P_{ext} [μW]
clk_i	140	2	...
add_i	90	...	...
a_i (per bit line)	60	...	...
b_i (per bit line)	60	...	...
output_o (per bit line)	0	...	...
Further nets	neglected in the context of this exercise		

Table 1: Power dissipated for driving the various nets of interconnect

	Dynamic Power [mW]	
	Switching Power	**Internal Power**
Adder	0.04	0.12
Multiplier	0.66	0.56
Output register + Mux	0.00	0.54

Table 2: Power dissipated by the various circuit blocks (@ 200 MHz).

Overall Dynamic Power Equation

We use the classical single-domain dynamic power approximation:

$$P_{\text{tot}} \approx \frac{1}{2} f_{\text{clk}} V_{\text{dd}}^2 \sum_{\text{cells}} (\alpha_{\text{int}}\, C_{\text{int}}); +; \frac{1}{2} f_{\text{clk}} V_{\text{dd}}^2 \sum_{\text{nets}} (\alpha_{\text{net}}\, C_{\text{ext}}).$$

Here, f_{clk} and V_{dd} are global; $\alpha_{\text{int}}, \alpha_{\text{net}}, C_{\text{int}}, \text{and } C_{\text{ext}}$ vary per cell/net.

Student Tasks

Student Task 1

Output waveform

- Based on the provided input waveforms (a_i, b_i, add_i), determine the resulting $output_o$ bit pattern for each clock cycle.
- Confirm the final 8-bit values at the rising edges of clk_i to verify correct addition or multiplication.

Student Task 2

Switching activities

- Assuming single-edge-triggered one-phase clocking, complete the node activity column in $Table 1$.

Student Task 3

Power spent for switching of nets

- You now have all the information required to calculate the switching powers associated with the various nets according to $Equation(2)$.
- Fill in the numbers into the last column of Table 1.

Student Task 4

Power dissipated within circuit blocks

- Now consider $Table2$. What is the main sink of power among the blocks listed there and how much does it dissipate?

Student Task 5

Combined dissipation

- Compiling all contributions from $Table1$ and $Table2$, how much power does the circuit dissipate internally, that is, with no load attached?
- Briefly discuss how the power might change if f_{clk} or V_{dd} were modified.

Student Task 6

Overall dissipation

- Suppose each output drives a load of $1pF$. What is the total power consumption now?

Power Estimation with EDA Tool

Required Files for Post-Layout Power Analysis in OpenROAD

The table below provides an overview of the files required to perform post-layout power analysis using OpenROAD. Each file is listed along with its format, purpose, and the design step in which it is generated.

File	Format	Description and Purpose	Generation Step
`croc.v`	Verilog	The gate-level netlist representing the logical structure of the design after synthesis or place-and-route. It lists all instantiated standard cells, macros, and nets. Required by OpenROAD to analyze logic connectivity and cell usage.	Output of synthesis or exported from OpenROAD after placement and routing.
`croc.def`	DEF (Design Exchange Format)	Describes the physical layout of the design, including standard cell and macro placements, routing, and power delivery network. Required to calculate wire-related capacitance and for IR drop analysis.	Exported from OpenROAD after detailed routing is complete.
`croc.sdc`	SDC (Synopsys Design Constraints)	Defines timing constraints such as clock periods, input/output delays, and false paths. Used during analysis to calculate switching frequency and timing windows.	Created manually or exported from synthesis tools and reused during physical design.
`*.lef`	LEF (Library Exchange Format)	Contains the physical layout data of standard cells, macros, and IO pads. Required to interpret cell geometries and placement during analysis.	Provided by the technology vendor (PDK). These files are static and used throughout the flow.
`*.lib`	Liberty	Characterizes standard cells and macros with timing and power data under specific operating conditions (PVT corners). Enables OpenROAD to compute internal, switching, and leakage power.	Provided by the technology vendor (PDK). These files are static and used throughout the flow.
`croc.vcd`	VCD (Value Change Dump)	Contains signal value transitions captured during simulation. Used to derive actual switching activity of nets and nodes for accurate dynamic power estimation.	Generated during post-layout gate-level simulation (e.g., using Verilator or QuestaSim).
`croc.spef`	SPEF (Standard Parasitic Exchange Format)	Parasitic RC extraction (net capacitances/resistances) for accurate dynamic power computation.	Extracted inside OpenROAD from the DEF file.

In the above table, two essential files have not yet been generated:

- `croc.vcd` — the Value Change Dump file, which records dynamic switching activity based on post-layout simulation.
- `croc.spef` — the Standard Parasitic Exchange Format file, which contains extracted resistance and capacitance values for routed nets.

In the following tasks, you will extract parasitics from the routed layout to generate the `.spef` file, and simulate the design to generate the `.vcd` file. Both files are required to perform accurate post-layout power analysis in OpenROAD.

Generate the SPEF File

Student Task 7

Generate the SPEF file

In this task, you will extract parasitic information from the post-layout design and generate a SPEF file using OpenROAD. This file contains resistance and capacitance values for all routed nets, which can be used in timing and power analysis.

To perform the extraction, we first start the OpenROAD shell, and load the technology files:

```
sh > cd openroad
sh > oseda -2025.01 openroad
OpenROAD > source scripts/init_tech.tcl
```

Then, in the OpenROAD shell, set the RC extraction rules file. This file defines how resistance and capacitance are extracted based on the metal stack used in IHP SG13G2:

```
OpenROAD > set extRules ./src/IHP_rcx_patterns.rules
```

Then load the physical layout of the design, including cell placements and routed interconnects. This is necessary for parasitic extraction to be based on actual wire lengths and geometry:

```
OpenROAD > read_def ./out_final/croc.def
```

Then defines a process corner (named `tt`) and associates it with the first RC extraction model (index 0). This step is necessary for tools that support multi-corner analysis:

```
OpenROAD > define_process_corner -ext_model_index 0 tt
```

Then, perform the actual extraction of parasitic resistances and capacitances using the model file you specified:

```
OpenROAD > extract_parasitics -ext_model_file $extRules
```

Finally, write the extracted parasitics to a file named `croc.spef` in SPEF format. This file can be reused in later steps for timing or power analysis:

```
OpenROAD > write_spef ./out_final/croc.spef
```

Generate the VCD File

In this exercise, we use the crc32 kernel to generate realistic activity for stimulus-based power analysis.

The program computes a 32-bit cyclic redundancy check (CRC) over a 256-byte data array using only integer operations. CRC is commonly used in hardware and software for data integrity verification, such as in communication systems or memory transfers.

Student Task 8

Prepare for VCD generation

Before running the simulation, open the testbench and netlist files to understand their structure.

- In the testbench file `rtl/tb_croc_soc.sv`, locate the macro `TARGET_NETLIST_OPENROAD`. This macro ensures that a dedicated module (`croc_chip`) is instantiated for post-layout simulation, matching the gate-level netlist exported by OpenROAD.

The testbench includes the following two lines for the VCD dump:

```
$dumpfile("croc.vcd"); — specifies the name of the output VCD file where signal transitions will be recorded.
$dumpvars(0, i_croc_soc); — instructs the simulator to record signal activity within the module instance i_croc_soc
(the DUT). The level 0 indicates that dumps all variables within module 'i_croc_soc' and in all sub-modules.
```

- In the netlist file `out_final/croc.v`, observe that it contains only structural Verilog (standard cell and macro instantiations), without any behavioural constructs. Ensure the top-level module matches what the testbench instantiates.

You do not need to modify the files, but ensure you understand how the testbench is configured to simulate the correct netlist and produce the correct VCD.

Student Task 9

Generate the VCD file

To perform a post-layout simulation and generate the VCD file, use the following command:

```
# Enter the simulation directory
sh > cd vsim

# Compile the testbench, netlist dut and technology libraries.
sh > questa-2019.3 vsim -c -do "source compile_post_pnr_netlist.tcl; source compile_tech.tcl; source
compile_tech_chip.tcl; exit"

# Then simulate the `crc32` benchmark and generate VCD switching activity:
sh > questa-2019.3 vsim +binary="../sw/bin_postsim/crc32.hex" -gui tb_croc_soc -t 1ns -voptargs=+acc \
     -suppress vsim-3009 -suppress vsim-8683 -suppress vsim-8386 \
     -do "run -all; exit"
```

Choosing the Proper VCD Range

When performing power analysis, it is critical to choose the simulation interval (start and end time) that accurately reflects active switching behavior.

Guidelines:

- **Exclude startup/reset phase**: The beginning of simulation may include reset or clock initialization, which is not representative of steady-state activity.
- **Focus on active workload phases**: Select the interval where the core is executing the actual benchmark or meaningful switching.
- **Avoid idle periods**: These can skew results toward artificially low dynamic power.

For example, in the testbench file `rtl/tb_croc_soc.sv`, you will find the VCD dump control block. In this case, the VCD dumping begins at **957177 ns** and continues for a duration of **58552 ns**, capturing only the active execution window of the kernel:

```
initial begin
    // wait until VCD dump start time
    #957177;

    $display("@%t | [VCD] Start dump", $time);
    $dumpfile("croc.vcd");
    $dumpvars(0, i_croc_soc);

    // keep dumping for duration = 1015729 - 957177 = 58552 ns
    #58552;

    $display("@%t | [VCD] Stop dump", $time);
    $dumpoff;
end
```

Once the SPEC and VCD files have been generated, you may proceed to the next section: performing power analysis in OpenROAD using this activity data.

Power Analysis: Generic Flow

Post-layout power analysis is a critical step in the digital design flow. It allows designers to estimate the actual power consumption of an integrated circuit based on its physical implementation, operating conditions, and functional behavior. Accurate power estimation helps assess the design's energy efficiency, ensure thermal reliability, and guide architectural or physical-level optimizations. There are two common methodologies for evaluating dynamic (switching-related) power:

- **Statistical Power Analysis**: Statistical power analysis estimates switching activity based on a fixed average toggle probability, typically assuming uniform and independent switching across all nodes. It is fast and does not require functional simulation, but does not account for actual data patterns or circuit state. It is most useful for early design exploration or as a reference baseline.
- **Stimuli-Based Power Analysis**: Stimuli-based power analysis uses signal activity captured during simulation, usually in the form of a VCD (Value Change Dump) file. The toggle information is mapped onto the post-layout design to compute real switching activity over time. This method is more accurate and reflects realistic circuit behavior under actual workloads or benchmarks, but requires a complete simulation setup and more runtime.

In the following tasks, you will first perform statistical power analysis using a fixed toggle activity, and then stimuli-based analysis using a VCD file obtained from post-layout gate-level simulation.

Power Analysis: Openroad Power Analysis Tool

OpenSTA (Open Static Timing Analyer) is the open-source engine used internally by OpenROAD to perform static timing and power analysis. It analyses timing paths, propagates delays, and evaluates signal activity across the entire design using standard file formats such as Liberty, Verilog, SPEF, and VCD. You may find its document here [OpenSTA_Doc (https://github.com/The-OpenROAD-Project/OpenSTA/blob/462853b3b5eb51be7e1c9c39ab16659cc7fd6acc/doc/OpenSTA.pdf)](https://github.com/The-OpenROAD-Project/OpenSTA/blob/462853b3b5eb51be7e1c9c39ab16659cc7fd6acc/doc/OpenSTA.pdf)

Statistical Power Analysis

Statistical power analysis is a fast, simulation-free method that estimates dynamic power based on a fixed average switching probability assigned to the input ports. This activity is then propagated through the logic gates using probabilistic models to estimate internal switching activity.

This method is useful in early-stage analysis or as a reference when simulation-based data is not yet available. It does not rely on a VCD file and avoids simulation overhead.

The underlying power calculation is performed by OpenSTA within OpenROAD using the following steps:

- Load the design netlist, parasitic data (SPEF), and timing constraints.
- Specify a toggle probability for all primary input ports using:

```
set_power_activity -input -activity <value>
```

- For known static inputs (e.g., `reset`), override their activity with zero using:

```
set_power_activity -input_port reset -activity 0
```

- Run the power analysis with:

```
report_power
```

OpenSTA uses these input toggle values and propagates switching probabilities to all internal pins to compute:

- Internal power
- Switching power (load capacitance-based)
- Leakage power

This method provides a consistent and quick estimation of power, although it may deviate from real workload behavior without input data correlation.

Student Task 10

Statistical Power Analysis

In this task, you will perform statistical power analysis by assigning a fixed switching activity to all input ports. These activities are then propagated through the logic network to estimate internal switching activity. This method does not require simulation data and provides a quick approximation of the dynamic power consumption.

Start OpenROAD and load the technology files using:

```
sh       > cd openroad
sh       > oseda -2025.01 openroad
OpenROAD > source scripts/init_tech.tcl
```

This command launches OpenROAD and preloads the technology LEF files and Liberty libraries for the typical (tt) and fast (ff) corners.

Then, in the OpenROAD shell, execute the following (for the detailed description of the power analysis related commands, you may refer to [OpenSTA_Doc (https://github.com/The-OpenROAD-Project/OpenSTA/blob/462853b3b5eb51be7e1c9c39ab16659cc7fd6acc/doc/OpenSTA.pdf)](https://github.com/The-OpenROAD-Project/OpenSTA/blob/462853b3b5eb51be7e1c9c39ab16659cc7fd6acc/doc/OpenSTA.pdf)):

```
# Load the gate-level netlist
read_verilog out_final/croc.v

# Link the design hierarchy using the top module name
link_design croc_chip

# Load timing constraints
read_sdc out_final/croc.sdc

# Load extracted parasitics
read_spef out_final/croc.spef

# Set uniform switching activity rate for all input ports, you may also replace <code>-input</code> with -global<code>
</code> to set activity rate for all pins
set_power_activity -input -activity 0.1

# Set known static inputs (e.g., reset) to zero activity
set_power_activity -input_port rst_ni -activity 0

# Generate the statistical power report for the typical corner
report_power -corner tt
```

Student Task 11

Statistical Power Analysis: different activity rate and corner

Try to use a different activity rate when `set_power_activity`.

Try to use a different corner (e.g. ff) when `report_power`.

What changed? Discuss your observations with the TA.

Stimuli-Based Power Analysis

Stimuli-based power analysis uses a Value Change Dump (VCD) file generated from post-layout simulation to annotate actual signal transitions in the design. This allows for realistic estimation of internal and switching power based on real workloads or testbenches, making it more accurate than statistical estimation.

Student Task 12

Stimuli-Based Power Analysis

Start OpenROAD and load the technology files:

```
sh       > cd openroad
sh       > oseda -2025.01 openroad
OpenROAD > source scripts/init_tech.tcl
```

Then, in the OpenROAD shell, execute the following commands:

```
# Load design and related files
read_verilog out_final/croc.v
link_design croc_chip
read_sdc out_final/croc.sdc
read_spef out_final/croc.spef

# Load the VCD file and define the simulation scope
read_vcd -scope tb_croc_soc/i_croc_soc ../vsim/croc.vcd

# Finally, you can generate the VCD-based power report
report_power -corner tt
```

Note on Clock Period Mismatch Warning (STA-1452). When reading a VCD file into OpenROAD, you may see warnings such as:

```
[WARNING STA-1452] clock clk_sys vcd period 225.530 differs from SDC clock period 12.500
```

These warnings are caused by the internal behavior of OpenSTA when estimating the clock period from the VCD data (you may check the code here OpenSTA_VcdReader1 (https://github.com/The-OpenROAD-Project/OpenSTA/blob/ce98bde188e7b330c86b913d9def696c950a4130/power/VcdReader.cc#L443)). Specifically, OpenSTA calculates the inferred clock period using the formula:

```
estimated_period = (last_timestamp_in_vcd) / (number_of_clock_transitions)
```

This calculation assumes the VCD dump starts at time zero, which is not the case in this exercise. Since we only dump a selected window of the simulation (e.g., starting at 957177 ns), the estimated period appears much longer than it actually is. This causes the toggle rate to be underestimated, and consequently, dynamic power is under-reported (you may check the code where it calculates the activity density here OpenSTA_VcdReader2 (https://github.com/The-OpenROAD-Project/OpenSTA/blob/ce98bde188e7b330c86b913d9def696c950a4130/power/VcdReader.cc#L409)).

How to Correct the Reported Power

We can scale the dynamic power based on the actual toggle duration to get a more accurate estimate of the power during the recorded interval.

Let:

- `T_start` = 957,177 ns
- `T_end` = 1,015,729 ns
- `Reported simulation window` = T_end - 0 = 1,015,729 ns
- `Actual toggle window` = T_end - T_start = 58,552 ns

Then the scaling factor is:

```
scaling_factor = T_end / (T_end - T_start)
               = 1,015,729 / (1,015,729 - 957,177)
               = 1,015,729 / 58,552 ≈ 17.35
```

So to estimate the actual dynamic power during the VCD capture window, we can multiply the reported dynamic power (from `report_power`) by 17.35

Note:

- This scaling applies only to dynamic power
- Leakage power is not affected by VCD window length
- The total power in OpenSTA is the sum of switching, internal, and leakage power

Student Task 13

Comparison of Statistical vs. Stimuli-Based Power Analysis

Compare the results obtained from the statistical power analysis (Task 10) and the stimuli-based power analysis using a VCD file (Task 12), and answer the following questions:

1. Which power component shows the largest difference between the two methods?

2. Which method do you think is more accurate for real workload estimation, and why?

IR Drop

Power analysis provides critical insights into the energy consumption of a circuit, enabling us to identify the most power-hungry components and optimize their design. However, understanding power consumption alone is not enough to ensure reliable operation. The distribution of power across the chip, particularly how voltage levels vary due to current flow and resistive losses, plays a vital role in circuit performance. This phenomenon, known as IR drop, can cause voltage levels to dip below acceptable thresholds, leading to timing failures and degraded functionality.

IR Drop Analysis with OpenROAD

IR drop analysis assesses voltage drops across the power distribution network (PDN) of an integrated circuit. Excessive IR drops can lead to functional failures or degraded performance. OpenROAD integrates PDNSim, an open-source static IR analyser, to perform this analysis, you may find its document here [PDNSim_Doc (https://openroad.readthedocs.io/en/latest/main/src/psm/README.html)](https://openroad.readthedocs.io/en/latest/main/src/psm/README.html).

Key features of PDNSim include:

- Reporting worst-case IR drops.
- Identifying areas with high current density.
- Detecting floating PDN stripes on power and ground nets.
- Generating SPICE netlists for detailed analysis.

Next, you will perform IR drop analysis using OpenROAD. IR drop refers to the voltage drop that occurs due to resistive losses in the power delivery network (PDN). This analysis allows you to identify areas of the chip where the power supply voltage drops significantly, potentially causing timing issues or functional failures.

Student Task 14

Launch OpenROAD GUI and Load the Design

Step 1: Start OpenROAD in GUI mode, and load the technology files:

```
sh       > cd openroad
sh       > oseda -2025.01 openroad -gui
OpenROAD > source scripts/init_tech.tcl
```

This launches OpenROAD with GUI support and loads technology and design settings.

Step 2: Load DEF and SDC File

```
OpenROAD > read_def out_final/croc.def
OpenROAD > read_sdc out_final/croc.sdc
```

- read_def: Loads the layout and placement information of your design.
- read_sdc: Loads timing constraints like clock periods and I/O delay.

Analyse VDD IR Drop – Supply Droop

In this task, you will perform IR drop analysis for the **VDD** net. The drop in supply voltage due to current flowing through the resistance of the power delivery network is called supply droop. Excessive droop can lower the effective voltage at critical cells, causing timing failures or even functional errors.

Student Task 15

IR Drop Analysis for VDD – Supply Droop

Step 1: Understand the `.loc` (Vsrc) File, which is located in the `openroad/src` folder

The .loc file specifies the location, size, and voltage of each power pad or C4 bump. It is required by PDNSim to determine where voltage sources are applied during IR drop analysis. You can find its document here [Vsrc_Doc (https://openroad.readthedocs.io/en/latest/main/src/psm/doc/Vsrc_description.html)](https://openroad.readthedocs.io/en/latest/main/src/psm/doc/Vsrc_description.html)

Example:

```
1515.0, -35.0, 70.0, 1.20
```

This indicates a bump centered at (1515.0 µm, -35.0 µm) with edge length 70 µm, supplying 1.2 V.

Step 2: Specify the activity rate for IR drop analysis

Switching activity is needed because it determines how much dynamic power a cell consumes, which in turn defines the current drawn from the power grid, directly affecting the IR drop calculation.

In OpenROAD, PDNSim relies on OpenSTA, the static timing analyzer we used earlier for power analysis, to estimate instance power. You can either use the `set_power_activity` command from Task 10 to assign a fixed switching activity, or use `read_vcd` from Task 12 to annotate actual switching activities from simulation data.

Step 3: Set Voltage and Analyse VDD

```
OpenROAD > set_pdnsim_net_voltage -net VDD -voltage 1.2
OpenROAD > analyze_power_grid -vsrc src/Vsrc_croc_vdd.loc -net VDD -corner tt
```

- set_pdnsim_net_voltage: Sets the VDD net voltage.
- analyze_power_grid: Performs IR drop analysis using Vsrc location and corner info.

Once the power grid analysis completes, you will get an IR drop report as sample below:

```
########## IR report #################
Net               : VDD
Corner            : tt
Supply voltage    : 1.20e+00 V
Worstcase voltage: 1.19e+00 V
Average voltage   : 1.19e+00 V
Average IR drop   : 8.88e-03 V
Worstcase IR drop: 1.07e-02 V
Percentage drop   : 0.89 %
######################################
```

- **Net**: The name of the power net being analyzed.
- **Corner**: The process-voltage-temperature (PVT) corner used for analysis.
- **Supply voltage**: The expected voltage value you assigned using `set_pdnsim_net_voltage`.
- **Worstcase voltage**: The lowest voltage observed at any node on the power net.
- **Average voltage**: The average voltage seen across all points (pins, stripes, and power taps) on the power net.
- **Average IR drop**: The average voltage drop across all nodes (equal to Supply Voltage – Average Voltage).
- **Worstcase IR drop**: The largest voltage drop across the net (equal to Supply voltage - Worstcase voltage).
- **Percentage drop**: The ratio of the worst-case IR drop to the supply voltage, expressed as a percentage: `(Worstcase IR Drop / Supply Voltage) × 100%`. Values over 5% are often critical.

Visualise IR Drop in GUI

After running the IR drop analysis for the power network, you can visualise the results using a heatmap in the Openroad GUI. This allows you to identify hot spots or vulnerable regions of the PDN.

Student Task 16

Visualise IR Drop in GUI

```
OpenROAD > gui::set_display_controls "Heat Maps/IR Drop" visible true
OpenROAD > gui::set_heatmap IRDrop Layer Metal1
OpenROAD > gui::set_heatmap IRDrop ShowLegend 1
```

- gui::set_display_controls: Makes the IR drop layer visible.
- gui::set_heatmap: Selects Metal1 and shows the legend with colour scale.

You should then be able to see the IR drop heat map like this:

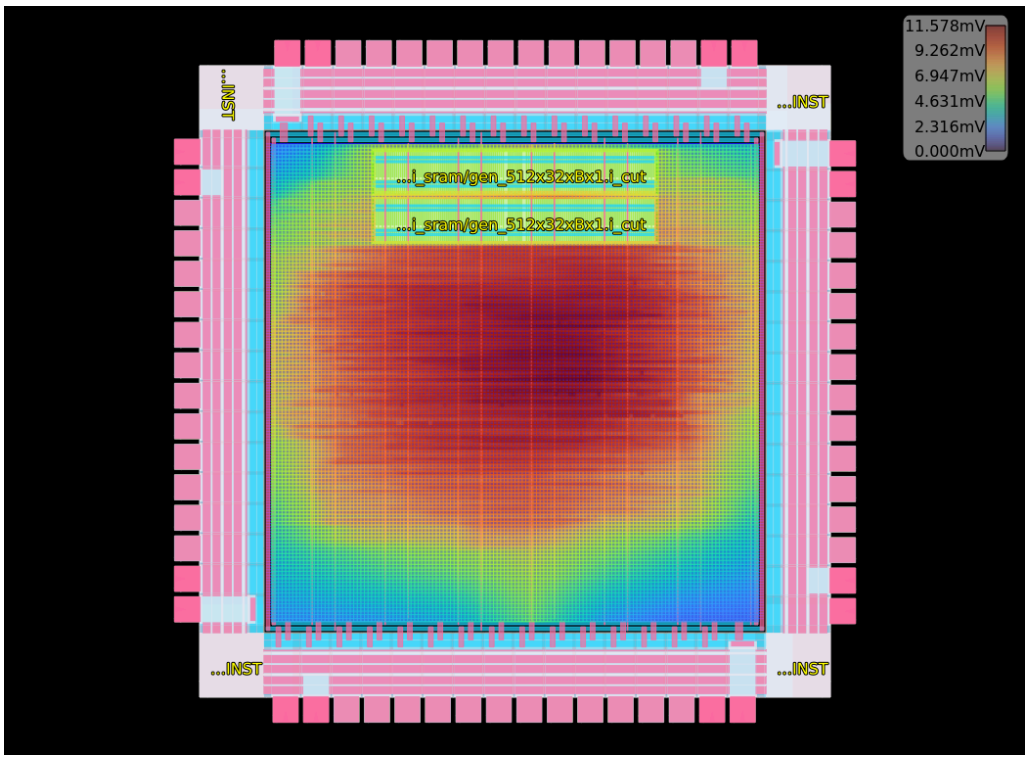

Figure 3: IR Drop Heatmap

Analyse VSS IR Drop – Ground Bounce

In this task, you will analyse the **VSS** net. The phenomenon of rising ground potential due to high current flow through ground lines is called ground bounce. It can cause false switching, logic level mismatch, and reduced noise margin.

Student Task 17

IR Drop Analysis for VSS – Ground Bounce

Set Ground Voltage and Analyse VSS

```
OpenROAD > set_pdnsim_net_voltage -net VSS -voltage 0
OpenROAD > analyze_power_grid -vsrc src/Vsrc_croc_vss.loc -net VSS -corner tt
```

- **set_pdnsim_net_voltage**: Sets expected voltage for the ground net (0 V).
- **analyze_power_grid**: Runs IR drop analysis on the ground network.

Then you can check the heat maps in the same way as in the previous task.

Explore the Impact of PDN Design on IR Drop

So far, we have analysed a well-designed power delivery network (PDN). In this section, you will explore four representative examples of poorly designed PDNs. Each variant isolates one type of PDN issue. You will:

- Run IR drop analysis on each variant
- Record the worst-case IR drop
- Describe the cause and impact of the observed PDN weakness

Student Task 18

IR Drop Evaluation of Faulty PDNs

Step 1: Reload Design for Each Variant

You can `exit` and relaunch OpenROAD with `oseda -2025.01 openroad -gui` before analysing a different PDN variant to ensure accurate results.

Step 2: IR drop analysis for Each Variant

For each variant in the table below, you can find their `.def` and `.sdc` files in the `openroad/out_[Variant Name]`, use the previous command sequence to do the IR drop analysis, and fill in the worst-case IR drop and observed PDN weakness.

Note You may analyse both the VDD and VSS nets, but it is recommended to analyse the VDD net first because IR-drop on the power supply directly impacts gate delay and chip timing, while VSS (ground) noise is typically less critical for digital circuits.

Variant Name	Worst-case IR Drop (mV)	Observed PDN Weakness
novstripe		
narrowstripe		
sparsestripe		
narrowring		

Please think about:

- Which PDN flaw caused the highest IR drop? Why?
- How could each PDN issue be mitigated?

Discuss your understanding with TAs.